

Hierarchical Overlap Graph

Bastien Cazaux^{a,b}, Eric Rivals^{a,b,*}

^a LIRMM, CNRS and Université de Montpellier (CC 477), 161 rue Ada, 34095 Montpellier Cedex 5, France

^b Institut de Biologie Computationnelle, CNRS and Université de Montpellier, 860 rue Saint Priest, 34095 Montpellier Cedex 5, France

ARTICLE INFO

Article history:

Received 1 March 2018

Received in revised form 13 September 2019

Accepted 14 September 2019

Available online 15 November 2019

Communicated by Łukasz Kowalik

Keywords:

On-line algorithms

Superstring

Assembly

Aho-Corasik

Data structures

ABSTRACT

Given a set of finite words, the Overlap Graph (OG) is a complete weighted digraph where each word is a node and where the weight of an arc equals the length of the longest overlap of one word onto the other (Overlap is an asymmetric notion). The OG serves to assemble DNA fragments or to compute shortest superstrings, which are a compressed representation of the input. The OG requires space that is quadratic in the number of words, which limits its scalability. The Hierarchical Overlap Graph (HOG) is an alternative graph that also encodes all maximal overlaps, but uses space that is linear in the sum of the lengths of the input words. We propose the first algorithm to build the HOG in linear space for words of equal length.

© 2019 Elsevier B.V. All rights reserved.

1. Introduction

DNA assembly problem arises in bioinformatics because DNA sequencing is unable to read out complete molecules, but instead yields partial sequences of the target molecule, called reads. Hence, recovering the whole DNA sequence requires to assemble those reads, that is to merge reads according to their longest overlaps (because of the high redundancy of sequencing strategies). DNA assembly comes down to building a digraph where sequences are nodes and arcs represent possible overlaps, and second to choosing a path that dictates in which order to merge the reads. The first proposed graph was the Overlap Graph [14,16]. Throughout this article, the input is $P := \{s_1, \dots, s_n\}$ a set of words. Without loss of generality, one considers that

P is substring free, that is no s_i is substring of s_j for $1 \leq i, j \leq n$. Let us denote the *norm* of P by $\|P\| := \sum_{i=1}^n |s_i|$.

Definition 1 (Overlap Graph). The *Overlap Graph* (OG) of P is a complete, directed graph, weighted on its arcs, whose nodes are the words of P , and in which the weight of an arc (u, v) equals the length of the maximal overlap from string u to string v .

In the OG (and in its variants such as the String Graph), an optimal assembly path can be computed by finding a Maximum Weighted Hamiltonian Path (which is NP-hard and difficult to approximate [3]). To build the graph, one has to compute the weights of the arcs by solving the so-called All Pairs Suffix Prefix overlaps problem (APSP) on P . Although, Gusfield has given an optimal time algorithm for APSP in 1992, APSP has recently regained attention due to innovation in sequencing technologies that allow sequencing longer reads. Indeed, solving APSP remains difficult in practice for large datasets. Several other optimal time algo-

* Corresponding author at: LIRMM, CNRS and Université de Montpellier (CC 477), 161 rue Ada, 34095 Montpellier Cedex 5, France.

E-mail addresses: Bastien.Cazaux@lirmm.fr (B. Cazaux), rivals@lirmm.fr (E. Rivals).

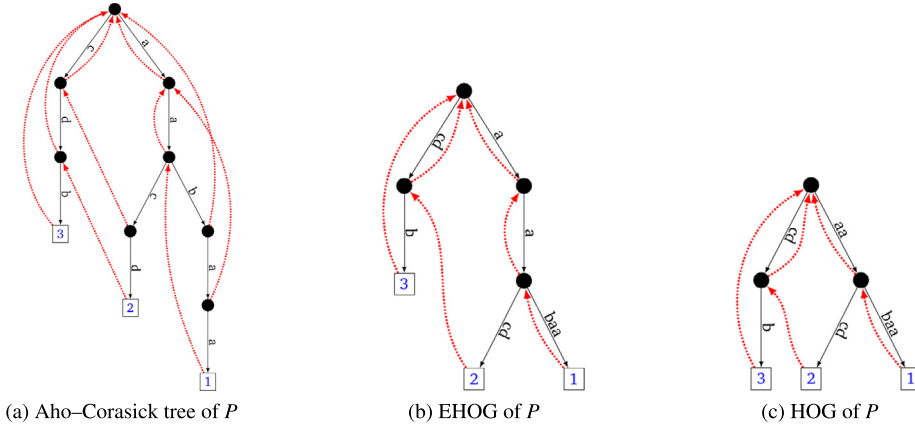


Fig. 1. Consider instance $P := \{aabaa, aacd, cdb\}$. (a) Aho-Corasick automaton of P with Failure links in dotted lines. (b) EHO of P - compared to (a) all nodes that were not overlaps were removed, and the arcs contracted. (c) HOG of P ; compared to (b) the node a has been removed, and some arcs contracted.

rithms that improve on practical running times have been described recently, e.g. [12,17].

The OG has several drawbacks. First, it is not possible to know whether two distinct arcs represent the same overlap. Second, the OG has an inherently quadratic size since it contains an arc for each possible (directed) pairs of words. Here, we present an alternative graph, called HOG, which represents all maximal overlaps and their relationships in terms of suffix and of prefix. Since the HOG takes a space linear in cumulated lengths of the words, it can advantageously replace the OG. Note that we already gave a definition of the HOG in [5]. Here, we proposed the first algorithm to build the HOG in linear space.

In DNA assembly, the de Bruijn Graph (DBG) is also used, especially for large datasets of short reads. For a chosen integer k , reads are split into all their k -long substrings (termed k -mers), which make the nodes of the DBG, and the arcs store only $(k-1)$ -long overlaps. Moreover, the relationship between reads and k -mers is not recorded. DBGs achieve linear space, but disregard many overlaps whose lengths differ from k . Hence, HOGs also represent an interesting alternative to DBGs.

1.1. Notation and definition of the Hierarchical Overlap Graph

We consider finite, linear of strings over a finite alphabet Σ and denote the empty string with ϵ . Let s be a string over Σ . We denote the length of s by $|s|$. For any two integers $i \leq j$ in $[1, |s|]$, $s[i, j]$ denotes the linear substring of s beginning at the position i and ending at the position j . Then we say that $s[i, j]$ is a *prefix* of s iff $i = 1$, a *suffix* iff $j = |s|$. A prefix (or suffix) s' of s is said *proper* if s' differs from s . For another linear string t , an overlap from s to t is a proper suffix of s that is also a proper prefix of t . We denote the longest such overlap by $ov(s, t)$. For A, B any two boolean arrays of the same size, we denote by $A \wedge B$ the boolean operation *and* between A and B .

Let $\mathcal{O}v^+(P)$ be the set of all overlaps between words of P . Let $\mathcal{O}v(P)$ be the set of **maximum** overlaps from a string of P to another string or the same string of P .

Let us define the *Extended Hierarchical Overlap Graph* and the *Hierarchical Overlap Graph* as follows. Both graphs are

directed graphs, made of nodes and arcs; thus, we use the notation (x, y) for an arc directed from node x toward node y .

Definition 2. The *Extended Hierarchical Overlap Graph* of P , denoted by $EHO(P)$, is the directed graph (V^+, E^+) where $V^+ = P \cup \mathcal{O}v^+(P)$ and E^+ is the set $E_g \cup E_f$ where

$$E_g := \{(x, y) \in (P \cup \mathcal{O}v^+(P))^2 \mid x \text{ is the longest proper prefix of } y\},$$

and

$$E_f := \{(x, y) \in (P \cup \mathcal{O}v^+(P))^2 \mid y \text{ is the longest proper suffix of } x\}.$$

The *Hierarchical Overlap Graph* of P , denoted by $HOG(P)$, is the directed graph (V, E) where $V := P \cup \mathcal{O}v(P)$, and E is the set $E_g \cup E_f$ where

$$E_g := \{(x, y) \in (P \cup \mathcal{O}v(P))^2 \mid x \text{ is the longest proper prefix of } y\},$$

and

$$E_f := \{(x, y) \in (P \cup \mathcal{O}v(P))^2 \mid y \text{ is the longest proper suffix of } x\}.$$

Clearly, $E_g \cap E_f = \emptyset$ as arcs in E_g represent a proper prefix relationship and those of E_f represent a proper suffix relationship; similarly, $E_g^+ \cap E_f^+ = \emptyset$. The subscript g reminds the *goto* transitions of the automaton, while the subscript f reminds the *Failure* links of the Aho-Corasick (AC) automaton [1].

Remark that $\mathcal{O}v(P)$ is a subset of $\mathcal{O}v^+(P)$. Both definitions are identical except that each occurrence of $\mathcal{O}v^+(P)$ in the $EHO(P)$ is replaced by $\mathcal{O}v(P)$ in the $HOG(P)$. Hence, $HOG(P)$ is somehow included in $EHO(P)$. Examples of EHO and HOG are shown in Figs. 1b, 1c.

1.2. Related works

To complete the Human genome project, improving the efficiency of OG based assemblers was compulsory. The prohibitive memory usage of OG led E. Myers to propose an alternative version of the OG (termed *String Graph*) where transitive arcs are removed [13]. Since then, several groups have proposed practical implementations of either the Overlap Graph [8] or the String Graph [15,10,2] that are incorporated in DNA assembly programs. These works aim at offering efficient assembly solutions by lowering the memory impact of the graph and improving the overall running time; for this sake they exploit either compressed data structures, like the FM-index [15] or Enhanced Suffix Arrays [10], or efficient probabilistic filters [2]. Noteworthy, Dinh and Rajasekaran showed how the OG can be compactly implemented in linear space [8]. These implementations are optimised for a given application. In contrast, in this work we define a different graph, the HOG, which encodes all maximal overlaps, but also additional information, that is their suffix-prefix relationships. Exploring the performance of various implementations of the HOG is beyond the scope of this paper. Note that a compact implementation of the Extended HOG, which serves as an index of overlaps, is presented, tested and compared in a preprint [4], and we comment on its efficiency in the conclusion.

In 1990, Ukkonen gave a linear time implementation of the greedy algorithm for finding a linear superstring of a set P [18]. This algorithm is based on the Aho–Corasick (AC) automaton [1]. In AC automaton, the input words are spelled out along a tree structure and each node represents a unique prefix of these words. The arcs of this tree are termed the *goto* transitions of the automaton, and the leaves represent the input words (Example in Fig. 1a: the tree structure is in black). The AC automaton contains additional arcs called, *Failure links*, which link two nodes say (x, y) if y is the longest suffix of x that is a node in the tree. Ukkonen characterised which nodes of the tree correspond to overlaps between words of P [18, Lemma 3]. In the EHOG, beyond the leaves, we keep only those nodes that are overlaps (Fig. 1b). From Ukkonen's lemma, it follows that $EHOG(P)$ is embedded in the Aho–Corasick automaton: the nodes of $EHOG(P)$ are a subset of those of the automaton, its arcs are contracted *goto* transitions or contracted *Failure links*.

Algorithm to build the EHOG. Given the Aho–Corasick automaton of P (whose underlying structure is a tree spelling out the words of P from the root – this tree is commonly called the *trie*), for each leaf, follow the chain of *Failure links* up to the root and mark all visited nodes. Thanks to [18, Lemma 3] all such nodes are overlaps. Another traversal of the AC automaton suffices to remove unmarked nodes and to contract the appropriate arcs. This algorithm takes $O(|P|)$ time. An involved algorithm to build a memory compact version of the EHOG is detailed in [4]. In the sequel, whenever we refer to the *tree* structure, it is the tree structure of the EHOG, we mean the tree defined by the *goto* arcs in the EHOG (which appears in black in Fig. 1b).

An algorithm for computing a shortest cyclic cover of a set of DNA sequences needs to build either the EHOG or HOG [5]. There, we stated mistakenly in Theorem 3 that the HOG could be built in $O(|P|)$ time, although we meant the EHOG. The construction algorithm uses the Generalised Suffix Tree of P to detect nodes representing overlaps as explained in [11].

The HOG is reminiscent of the “Hierarchical Graph” of [9], where the arcs also encode inclusion between substrings of the input words. However, in the Hierarchical Graph, each arc extends or shortens the string by a single symbol, making it larger than the HOG.

2. Construction algorithm for the HOG

All internal nodes of $EHOG(P)$ are overlaps between words of P , while those of $HOG(P)$ are *maximal* overlaps between words of P . Given the EHOG of P , to build $HOG(P)$ we need to discard nodes that are not maximal overlaps and to merge the arcs entering and going out of such nodes. This processing can be performed in linear time on the size of $EHOG(P)$ provided that the nodes of $HOG(P)$ are known. We present Algorithm 1, which recapitulates this construction procedure. Once $EHOG(P)$ is built, each of its internal nodes u is equipped with a list $R_l(u)$, whose meaning is explained below. Then, Algorithm 1 calls the key procedure $\text{MarkHOG}(r)$ on line 4 to mark the nodes of $HOG(P)$ in a global boolean array denoted bHog , which we store in a *bit vector*. The procedure for MarkHOG is given in Algorithm 2. Once done, it contracts $EHOG(P)$ in place to obtain $HOG(P)$.

Algorithm 1: HOG construction.

Input : P a substring free set of words
Output: $HOG(P)$; **Variable**: bHog a bit vector of size $|EHOG(P)|$
1 build $EHOG(P)$
2 set all values of bHog to False
3 traverse $EHOG(P)$ to build $R_l(u)$ for each internal node u of $EHOG(P)$
4 run $\text{MarkHOG}(r)$ where r is the root of $EHOG(P)$
5 Contract($EHOG(P)$, bHog)
// Procedure Contract traverses $EHOG(P)$ to
discard nodes that are not marked in bHog and
contract the appropriate arcs

Meaning of $R_l(u)$ For any internal node u , $R_l(u)$ lists the words of P that admit u as a suffix. Formally stated: $R_l(u) := \{i \in \{1, \dots, |P|\} : u \text{ is suffix of } s_i\}$. As we need to iterate over $R_l(u)$, it is convenient to store it in a list of integers. A traversal of $EHOG(P)$ allows to build a list $R_l(u)$ for each internal node u as stated in [18]. Remark that, while our algorithm processes $EHOG(P)$, Ukkonen's algorithm processes the Aho–Corasick automaton, in which $EHOG(P)$ is embedded. The cumulated sizes of all R_l is linear in $|P|$ (indeed, internal nodes represent different prefixes of words of P and have thus different begin/end positions in those words).

Nodes of the EHOG and overlaps The following proposition states an important property of EHOG nodes in terms of overlaps. It will allow us not to consider all possible pairs in $P \times P$, when determining maximum overlaps.

Proposition 1. Let u be an internal node of $EHOG(P)$ and let $i \in R_l(u)$. Then, for any word s_j whose leaf is in the subtree of u , we have $ov(s_i, s_j) \geq |u|$.

Proof. Indeed, as u belongs to $EHOG(P)$, we get that u is an overlap from s_i to s_j . Thus, their maximal overlap has length at least $|u|$. $\square$

Description of Algorithm 2 We propose Algorithm 2, a recursive algorithm that determines which nodes belong to $\mathcal{O}v(P)$ while traversing $EHOG(P)$ in a depth first manner, and marks them in $bHog$. At the end of the algorithm, for any node w of $EHOG(P)$, the entry $bHog[w]$ is **True** if and only if w belongs to $HOG(P)$, and **False** otherwise.

By the definition of a HOG, the leaves of the $EHOG(P)$, which represent the words of P , also belong to $HOG(P)$ (hence, line 3).

The goal is to distinguish among internal nodes those representing maximal overlaps (i.e., nodes of $\mathcal{O}v(P)$) of at least one pair of words. We process internal nodes in order of decreasing word depth. By Proposition 1, we know that for any i in $R_l(u)$, s_i overlaps any s_j whose leaf is in the subtree of u . However, u may not be the maximal overlap for some pairs, since longer overlaps may have been found in each subtree. Indeed, u can be a maximal overlap from s_i onto some s_j , if and only if for any child v of u , s_i has not already a maximal overlap with the leaves in the child's subtree. Hence, to check this, we compute the C vector for each child by recursively calling **MarkHOG** for each child, and merge them with a boolean **and** (line 7). We get invariant #1:

$C[w]$ is **True** iff for any leaf l in the subtree of u

the pair $ov(w, l) > |u|$.

Then, we scan $R_l(u)$, for each word w if $C[w]$ is **False**, then for at least one word s_j , the maximum overlap of w onto s_j has not been found in the subtree. By Proposition 1, u is an overlap from w onto s_j , and thus we set both $C[w]$ and $bHog[u]$ to **True** (lines 9–11). Otherwise, if $C[w]$ is **True**, then it remains so, but then $bHog[u]$ remains unchanged. This yields Invariant #2:

$C[w]$ is **True** iff for any leaf l in the subtree of u

the pair $ov(w, l) \geq |u|$.

This ensures the correctness of **MarkHOG**.

The complexity of Algorithm 1 is dominated by the call of the recursive algorithm **MarkHOG**(r) in line 4, since computing $EHOG(P)$, building the lists $R_l(\cdot)$ and contracting the $EHOG$ into the HOG (line 5) all take linear time in $|P|$.

How many simultaneous calls of **MarkHOG** can there be? Each call uses a bit vector C of length $|P|$, which impacts the space complexity. The following proposition helps answering this question.

Proposition 2. Let u, v be two nodes of $EHOG(P)$. If u and v belong to distinct subtrees, then **MarkHOG**(u) terminates before **MarkHOG**(v) begins, or vice versa.

Algorithm 2: **MarkHOG**(u);.

```

Input :  $u$  a node of  $EHOG(P)$ 
Output:  $C$ : a boolean array of size  $|P|$ 
1 if  $u$  is a leaf then
2   set all values of  $C$  to False
3    $bHog[u] := \text{True}$ 
4   return  $C$ 
   // Cumulate the information for all children of
   //  $u$ 
5  $C := \text{MarkHOG}(v)$  where  $v$  is the first child of  $u$ 
6 foreach  $v$  among the other children of  $u$  do
7    $C := C \wedge \text{MarkHOG}(v)$ 
   // Invariant 1:  $C[w]$  is True iff for any leaf  $s_j$ 
   // in the subtree of  $u$  the pair  $ov(w, s_j) > |u|$ 
   // Process overlaps arising at node  $u$ : traverse
   // list  $R_l(u)$ 
8 for node  $x$  in the list  $R_l(u)$  do
9   if  $C[x] = \text{False}$  then
10     $bHog[u] := \text{True}$ 
11     $C[x] := \text{True}$ 
   // Invariant 2:  $C[w]$  is True iff for any leaf  $s_j$ 
   // in the subtree of  $u$  the pair  $ov(w, s_j) \geq |u|$ 
12 return  $C$ 

```

Hence, the maximum number of simultaneously running **MarkHOG** procedures is bounded by the maximum (node) depth of $EHOG(P)$, which is itself bounded by the length of the longest word in P . Now, consider the amortised time complexity of Algorithm 2 over all calls. For each node, the corresponding C vector is computed once and merged once during the processing of his parent. Moreover, if amortised over all calls of **MarkHOG**, processing all the lists R_l for all nodes takes linear time in $|P|$. Altogether all calls of Algorithm 2 require $O(|P| + |P|^2)$ time. The space complexity sums the space occupied by $EHOG(P)$, i.e. $O(|P|)$, and that of the C arrays. Now, the maximum number of simultaneously running calls of **MarkHOG** is the maximum number of simultaneously stored C vectors. This number is bounded by the maximum number of branching nodes on a path from the root to a leaf of $EHOG(P)$, which is smaller than the minimum among $\max\{|s| : s \in P\}$ and $|P|$. Hence, the space complexity is $O(|P| + |P| \times \min(|P|, \max\{|s| : s \in P\}))$.

Theorem 3. Let P be a set of words. Then Algorithm 1 computes $HOG(P)$ using $O(|P| + |P|^2)$ time and $O(|P| + |P| \times \min(|P|, \max\{|s| : s \in P\}))$ space. If all words of P have the same length, then the space complexity is $O(|P|)$.

An example of HOG construction Consider the instance $P := \{tattatt, ctattat, gtattat, cctat\}$; the graph $EHOG(P)$ is shown Fig. 2a. Table 2b traces the execution of Algorithm 2 for instance P . It describes for each internal node (leaves not included), the list R_l , the computation of C vector, the final value of $bHog$ and the list of specific pairs of words for which the corresponding node is a maximum overlap. We write $(x/y, z)$ as a shortcut for pairs (x, z) and (y, z) . Another example is given in the web [appendix](http://www.lirmm.fr/~rivals/res/superstring/hog-art-appendix.pdf) (URL: <http://www.lirmm.fr/~rivals/res/superstring/hog-art-appendix.pdf>).

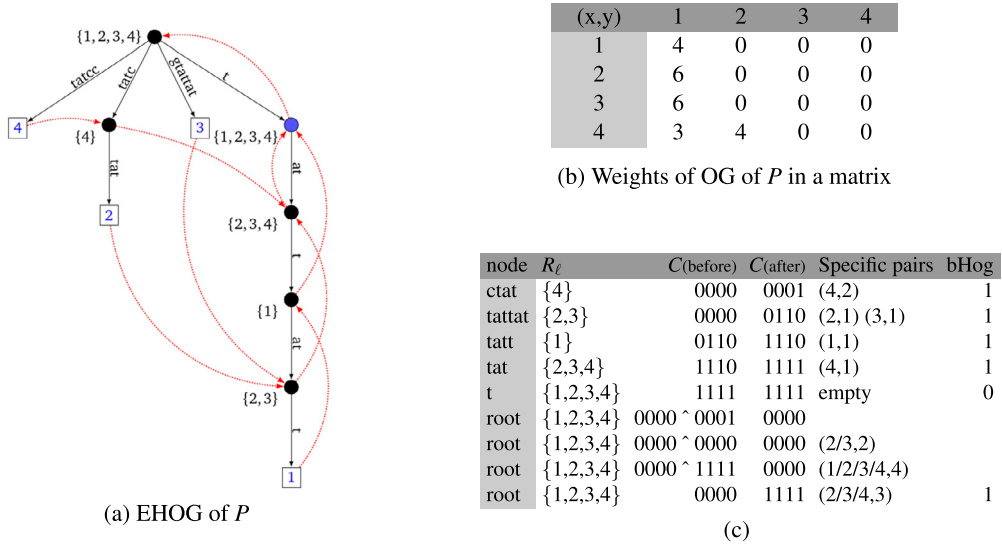


Fig. 2. (a) EHOg for instance $P := \{tattat, ctattat, gtattat, cctat\}$. goto transitions appear in black arcs, Failure Links in dotted red arcs. For each internal node, the list R_ℓ is given between brackets. (b) Overlap Graph of P given in matrix form. (c) Trace of Algorithm 2. For each internal node are shown: the word it represents, R_ℓ , the bit vector C when before and after the node is processed, bHog, and the pairs for which it is a maximum overlap. The node t is the only internal node which is not a maximal overlap for some pair, and indeed bHog is set to 0. The computation for the root node shows each and between C vectors from the children on four lines. (For interpretation of the colours in the figure(s), the reader is referred to the web version of this article.)

Querying for a maximal overlap Let x, y be two words of P , and assume one can directly access the nodes representing x and y in the HOG. Since nodes represent strings, one can define the *string-depth* of a node as the length of the string it represents, and store it in the node. Assume that in each node w , one stores: 1/ in an attribute denoted $\text{Parent}[w]$, its parent by the goto arcs if it exists and NIL otherwise; 2/ and in $\text{FL}[w]$, its child according to its Failure Link if it exists and NIL otherwise. Now, one can find the node representing $\text{ov}(x, y)$ in the HOG as follows. Initialise two variables n_x and n_y to point to the nodes representing x and y , respectively. Until either n_x and n_y are identical, or one of them equal NIL, repeat: if depth of n_x is larger than that of n_y , do: $n_x := \text{FL}[x]$; else do: $n_y := \text{Parent}[y]$. Finally, if $n_x = n_y$, return n_x , or NIL otherwise. Clearly, this procedure takes at most $O(|x| + |y|)$ time and $O(1)$ additional space.

Performance experiments run with an EHOg implementation (not a HOG) confirm that this procedure is reasonably efficient in practice [4]. However, note that this implementation is designed to support complex queries such as: given a word x compute the other words y such that $|\text{ov}(x, y)|$ is maximal among all possible pairs (x, z) , where x, y, z denote three words of P .

3. Difference in number of nodes between EHOg and HOG

The EHOg and HOG differ by definition, but the nodes of the HOG are a subset of the nodes of the EHOg. For instance, a small alphabet (e.g. DNA) favours multiple overlaps and tend to increase the number of nodes that are proper to the EHOg, while for natural languages HOG and EHOg tend to be equal. Can this difference in number of nodes be substantial? First, in our web [appendix](#)

(URL: <http://www.lirmm.fr/~rivals/res/superstring/hog-art-appendix.pdf>), we give an instance for which EHOg and HOG are identical. Below, we exhibit a family of instances such that in the limit the ratio between the number of nodes of EHOg versus HOG tends to infinity when $\|P\|$ tends to infinity, while the number of input words remains bounded.

Consider a finite alphabet, say $\Sigma = \{a, c, g, t\}$. Let s be any word formed by a permutation of Σ . Here, s can be $s := acgt$. The length of s is the cardinality of Σ .

Let z be a positive integer. We build the following instance P_z by taking:

- the word w made of the concatenation of z copies of word s ,
- and $(|\Sigma| - 1)$ other words, which are $(|\Sigma| - 1)$ cyclic shifts of w , denoted $w_1, \dots, w_{(|\Sigma|-1)}$.

For a letter α and a word v , the (first) cyclic shift of the word αv is $v\alpha$. With this construction, w_1 is the cyclic shift of w , w_2 is the cyclic shift of w_1 , ..., and w is the cyclic shift of $w_{(|\Sigma|-1)}$. Note that $|P_z| = |\Sigma| = |s|$ and that $\|P_z\| = |P_z| \times |w| = |P_z| \times z \times |s| = |P_z|^2 \times z$. For an instance P_z ,

- the $\text{EHOg}(P_z)$ contains exactly as many nodes as the Aho-Corasick automaton contains states, that is $\|P_z\|$ nodes (leaves included). Indeed, all internal nodes are overlaps.
- The $\text{HOG}(P_z)$ contains $|P_z|^2$ internal nodes and $|P_z|$ leaves.

Let us denote the number of nodes of the EHOg and of the HOG respectively by $|\text{EHOg}(P_z)|$ and $|\text{HOG}(P_z)|$. Then the ratio

$$\begin{aligned} \frac{|EHOG(P_z)|}{|HOG(P_z)|} &= \frac{||P_z||}{|P_z|^2 + |P_z|} = \frac{z \times |P_z|^2}{|P_z|^2 + |P_z|} \\ &= \frac{z}{1 + \frac{1}{|P_z|}} \xrightarrow{z \rightarrow \infty} +\infty. \end{aligned}$$

4. Conclusion

The Hierarchical Overlap Graph (HOG) is a compact alternative to the Overlap Graph (OG) since it also encodes all maximal overlaps as the OG, but in a space that is linear in the norm of P . In addition, the HOG records the suffix- and the prefix-relationship between the overlaps, while the OG lacks this information, which is useful for computing greedy superstrings [18,7]. Because the norm of P can be large in practice, it is thus important to build the HOG also in linear space, which our algorithm achieves if all words have the same length.

For constructing the HOG, Algorithm 1 takes $O(||P|| + |P|^2)$ time. Whether one can compute the HOG in a time linear in $||P|| + |P|$ remains open. An argument against this complexity is that all the information needed to build the Overlap Graph is encoded in the HOG.

For any instance, the HOG can be obtained from the EHOG by deleting nodes and contracting arcs. In an available preprint, we proposed and evaluated a data structure implementing the EHOG, which turns out to be efficient and scalable [4]. Moreover, we used this structure to compute approximations of the shortest superstring of a set of words in practice [6]; experiments with instances containing up to 1.5 billion words of length 100 each can run on a desktop computer with 32 gigabytes of memory, further demonstrating its scalability and efficiency. Reasonably, a HOG implementation similar to the EHOG one should be practical as well.

For some applications like DNA assembly, it is valuable to compute approximate rather than exact overlaps. The approach proposed here does not easily extend to approximate overlaps. Some algorithms have been proposed to compute OG with arcs representing approximate overlaps, where approximation is measured by the Hamming or the edit distance [19].

Declaration of competing interest

The authors declare not having any conflict of interest.

Acknowledgements

This work is supported by the Institut de Biologie Computationnelle ANR (ANR-11-BINF-0002), and Défi

MASTODONS C3G from CNRS. We thank A. Bergeron and K. Swenson for proof reading the abstract.

References

- [1] Alfred V. Aho, Margaret J. Corasick, Efficient string matching: an aid to bibliographic search, *Commun. ACM* 18 (1975) 333–340.
- [2] Ilan Ben-Bassat, Benny Chor, String graph construction using incremental hashing, *Bioinformatics* 30 (24) (09 2014) 3515–3523.
- [3] Avrim Blum, Tao Jiang, Ming Li, John Tromp, Mihalis Yannakakis, Linear approximation of shortest superstrings, in: *ACM Symposium on the Theory of Computing*, 1991, pp. 328–336.
- [4] Rodrigo Cánovas, Bastien Cazaux, Eric Rivals, The compressed overlap index, *CoRR*, arXiv:1707.05613 [cs.DS], 2017.
- [5] Bastien Cazaux, Rodrigo Cánovas, Eric Rivals, Shortest DNA cyclic cover in compressed space, in: *Data Compression Conference, IEEE Computer Society Press*, 2016, pp. 536–545.
- [6] Bastien Cazaux, Samuel Juhel, Eric Rivals, Practical lower and upper bounds for the shortest linear superstring, in: *17th Int. Symp. on Experimental Algorithms (SEA)*, in: *LIPIcs*, vol. 103, 2018, 18.
- [7] Bastien Cazaux, Eric Rivals, A linear time algorithm for shortest cyclic cover of strings, *J. Discret. Algorithms* 37 (2016) 56–67.
- [8] Hieu Dinh, Sanguthevar Rajasekaran, A memory-efficient data structure representing exact-match overlap graphs with application for next-generation DNA assembly, *Bioinformatics* 27 (14) (06 2011) 1901–1907.
- [9] Alexander Golovnev, Alexander S. Kulikov, Ivan Mihajlin, Solving SCS for bounded length strings in fewer than 2^n steps, *Inf. Process. Lett.* 114 (8) (2014) 421–425.
- [10] Giorgio Gonnella, Stefan Kurtz, Readjoinder: a fast and memory efficient string graph-based sequence assembler, *BMC Bioinform.* 13 (1) (May 2012) 82.
- [11] Dan Gusfield, Gad M. Landau, Baruch Schieber, An efficient algorithm for the all pairs suffix-prefix problem, *Inf. Process. Lett.* 41 (4) (1992) 181–185.
- [12] Jihyuk Lim, Kunsoo Park, A fast algorithm for the all-pairs suffix-prefix problem, *Theor. Comput. Sci.* 698 (2017) 14–24.
- [13] Eugene W. Myers, The fragment assembly string graph, *Bioinformatics* 21 (Suppl. 2) (2005) 79–85.
- [14] Hannu Peltola, Hans Söderlund, Jorma Tarhio, Esko Ukkonen, Algorithms for some string matching problems arising in molecular genetics, in: *IFIP Congress*, 1983, pp. 59–64.
- [15] Jared T. Simpson, Richard Durbin, Efficient construction of an assembly string graph using the FM-index, *Bioinformatics* 26 (12) (2010) 06, i367–i373.
- [16] Jorma Tarhio, Esko Ukkonen, A greedy approximation algorithm for constructing shortest common superstrings, *Theor. Comput. Sci.* 57 (1988) 131–145.
- [17] William H.A. Tustumi, Simon Gog, Guilherme P. Telles, Felipe A. Louza, An improved algorithm for the all-pairs suffix-prefix problem, *J. Discret. Algorithms* 37 (2016) 34–43.
- [18] Esko Ukkonen, A linear-time algorithm for finding approximate shortest common superstrings, *Algorithmica* 5 (1990) 313–323.
- [19] Niko Välimäki, Susana Ladra, Veli Mäkinen, Approximate all-pairs suffix/prefix overlaps, *Inf. Comput.* 213 (2012) 49–58.